Manual: Reversi

Overview

Minimax is a predictive game playing technique. A minimax agent is ***perfectly rational*** – it makes the optimal, rational decision based on the assumption that the opponent is also a perfectly rational agent. This is done by trying to maximize the agent's position by predicting that the opponent will try to minimize that same position.

In this exercise, students will implement a Minimax behavior for the game Reversi (Othello) with a ***depth-limit***. The complete Minimax algorithm traces all possible sequences of actions from the current game state to all possible game outcomes; as such, a complete Minimax search is impractical in typical game scenarios. As such, most such algorithms used in practice employ a depth-limit; the algorithm will search game states only a limited number of actions from the current state, then use a heuristic evaluation as an estimate of the state's value.

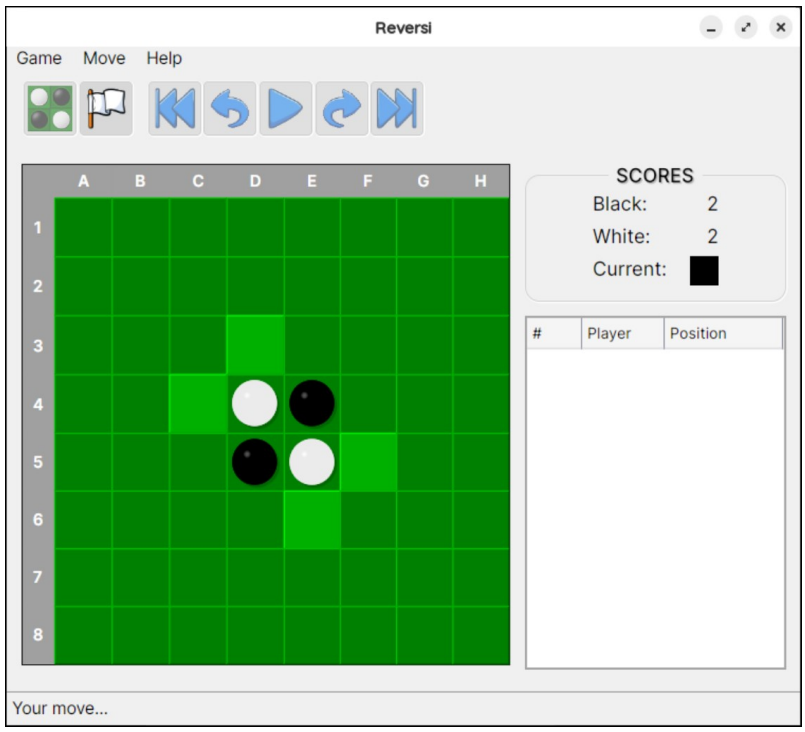

Figure 1. Reversi C# Application

Rules of Reversi

Reversi (**Figure 1**) is a two-player strategy game played on an 8x8 board using discs, white on one side and black on the other. One player plays the discs black side up while their opponent plays the discs white side up. The object of the game is to place your discs on the board to outflank one's opponent's discs, flipping them over to one's own color. The player with the most discs on the board at the end of the game wins.

Game Play

Every game starts with four discs placed in the center of the board, two of each color diagonally from each other. Players take turns making moves, with black making the first move. A move consists of a player placing a disc of their color on the board. The disc must be placed to outflank one or more opponent discs, which are then flipped over to the current player's color. Outflanking one's opponent means to place one's disc such that it traps one or more of one's opponent's discs between two discs of one's own color along a horizontal, vertical or diagonal line through the board square.

Move Forfeit

If a player cannot make a legal move, they forfeit their turn and the other player moves again; this is also known as passing a turn. Note that a player may not forfeit their turn voluntarily. If a player can make a legal move on their turn, they must do so.

End of the Game

The game ends when neither player can make a legal move. This includes cases when there are no more empty squares on the board or when one player has flipped over all their opponent's discs – a situation commonly known as a wipeout. The player with the most discs of their color on the board at the end of the game wins. The game is a draw if both have the same number of discs on the board.

Structure

In this exercise, students will implement Minimax for Reversi using the interfaces defined in this section.

Development Interface

These classes, representing Reversi game elements, are in the `Uwu.Games.Reversi` namespace.

Reversi.Player

This enumeration class represents a player in the Reversi game. This is used to identify the players and also to mark game board locations, with an additional value for unfilled spaces:

```
public enum Player { Black = -1, Empty = 0, White = 1 }
```

Reversi.Move

This class represents a move in the game. It holds the row and column of the move, as well as a value that can be assigned to the move's rank (according to an evaluation function or other algorithm). It has these properties:

`public int` **`Row`**	*(Read-only)*	The move's row
`public int` **`Column`**	*(Read-only)*	The move's column
`public int` **`Rank`**	*(Read-write)*	The move's rank (assignable for evaluation)

Reversi.State

This class represents the game board and current player at a given moment in time.

Read-Only / Constant Attributes

`public const int` **`DIMENSION_COUNT, GRID_SIZE`**
In dictates the number of dimensions of the board (always 2) and the length/width (always 8). These values can also be found in the ***`Dimensions`*** property (whose length is the number of dimensions).

Relevant Properties & Operators

`public int` **`Score`** *(Read-only)*
Returns the current score based purely on the number of pieces. ***This is not an evaluation function!***

`public Player?` **`Current`** *(Read-only)*
identifies the current player at this state in the game. **NOTE:** The object does not automatically change this; it is up to the student to make sure this value stays up to date!

`public Player` **`this`**`[int row, int column]`
Returns the entry stored at this board location, with a value representing a player or **`Player.Empty`**.

Constructors

`public State()`
Allocates a Reversi game state object with an empty board.

`public State(State source)`
Allocates a new **`Reversi.State`** object, then copies the referenced **`source`**'s entries (including current player) into that new state.

Relevant Methods

`public bool HasAnyValidMove(int color)`
This method will determine if the player of the specified `color` has and valid moves to make. This method is useful in determining whether a player must forfeit a turn when predicting moves.

`public bool IsTerminalState()`
Returns **`true`** if this state is an endgame state (i.e., if the game is over and no further moves can be made).

`public bool IsValidMove(int color, int row, int column)`
Returns **`true`** if placement of a disc at position (`row, column`) by the player specified by `color` is valid.

`public void MakeMove(Move move)`
Places a disc for the *current player* (according to the game state) and flips any outflanked opponent pieces to `color`. Note: for performance reasons, this method **does not** check that the move in question is valid.

`public static Move NewMove(int row, int column)`
Allocates and returns a **`Move`** object representing a move to be considered or made at the designated location.

`public Player Next(Board board)`
Use this method instead of the copy constructor (thus avoiding allocation) to copy `board` over this object.

`public void setCurrent(Player player)`
Updates the current player (whose turn it is) for this game state.

Assignment Interface

Students will implement the AI player behavior, matching the specification outlined in this section. The classes outlined are found in the **`Uwu.Games.Reversi.Student`** namespace.

Minimax

Objects of this class represent a unique instance of an AI agent / its game behavior. While implementations must have the methods listed as required, students are also encouraged to write their own private helper methods to deconstruct the problem in order to construct a robust gameplaying agent.

Required Methods

`public Minimax()`
Constructor; creates a new Minimax AI agent behavior.

`public Move Run(State gameState, int lookAheadDepth)`
Implementation of Minimax. Determines the best move for the player specified as the current player for the given `gameState`, looking ahead the number of steps indicated by `lookAheadDepth`. This method will not be called unless there is at least one possible move for the player.

Recommended Methods

```
private int Evaluate(State gameState)
```

Returns a strategic value estimate for `gameState` based on position and color of pieces. It is recommended that students implement a method with this functionality to evaluate board states at the lookahead depth cutoff.

Testing

It is important for students to carefully test their agents before submission. Please read the following guidelines.

Reproducibility

In production, we would ordinarily prefer moves to have a random offset, as this would make the agent less predictable to players. However, for testing purposes, it is important that behavior is reproducible – and this includes case tests used to evaluate student exercises. As such, **it is crucial** that your agent consider board tiles in the following order:

```
for row in range(0, 8):
  for column in range(0, 8):
    # Consider move at location (row, column)
```

If your agent does not use this ordering, it is likely to fail to meet or exceed the example AI, and therefore will be marked down accordingly!

Evaluation

The **`Example.Minimaximus`** class provides a method so students may incrementally build their agents:

```
public static int EvaluateTest(State gameState)
```

This method provides an evaluation of the game board state which is also printed to the console. ***This is also the heuristic that our example AI uses.*** Please note that this method is **only for testing** and should not be left in student code when submitted; exercise submissions which use this method in the final implementation **will be marked zero**.

The **`EvaluateTest`** method determines a board's strategic value using the following criteria:

1. A square's value is determined by the color of the piece contained: White = 1, Black = -1, Empty = 0.
2. If a square is in a corner, its value is multiplied by 100.
3. If a square is not in a corner but is on the side of the board, its value is multiplied by 10.
4. One the value for each square is identified, all values are added together.
5. If the game is over, value increases by 10,000 if the score is positive / decreases by 10,000 if negative.

The student's implementation should perform at least well as the example AI to receive full credit in all test cases. All tests will be done with the student player as the color **black**. For example, if the Example Intermediate vs Example Beginner results in black winning 33 to 30, then Student Intermediate vs Example Beginner should also result in a win for black with a ratio of 110% (33 to 30 or 22 to 20, for example). At minimum, these tests will be run:

- Beginner (black) vs. Beginner (white)
- Intermediate (black) vs. Beginner (white)
- Intermediate (black) vs. Intermediate (white)
- Advanced (black) vs. Beginner (white)
- Advanced (black) vs. Intermediate (white)
- Advanced (black) vs. Advanced (white)

Submissions

Students will submit a **zip file** containing the following file(s) at the end of this exercise on Canvas:

- Student.Minimax.cs

Place them in the **root directory** of your zip file, not in a subdirectory. Do not submit any other files.